GhostLeaks Documentation

Full Source Code Documentation | Ritesh Kumar

Breach Monitoring Web App — Full Source Code
Built with Python (Flask), PostgreSQL, and HaveIBeenPwned API

1. Overview

GhostLeaks is a cybersecurity SaaS tool that checks whether an email address or password has been exposed in known data breaches. It uses the k-anonymity model (the same approach used by HaveIBeenPwned) so that full passwords or hashes are never transmitted or stored in plaintext.

This package includes the complete, production-ready source code for GhostLeaks, along with setup instructions so you can deploy it on your own server or customize it for your own use case.

Key Features

- Email & Password Breach Check — checks credentials against HaveIBeenPwned and BreachDirectory APIs
- Bulk CSV Scanning — upload a list of emails and scan them in one go
- PDF Report Generation — generates downloadable breach reports (via ReportLab)
- User Authentication — secure login/signup system (passwords hashed with bcrypt)
- Scan History Dashboard — users can view past scan results
- Continuous Monitoring — users can add an API key to enable automatic daily re-checks, with alerts if a new breach is found
- Razorpay Payment Integration — ready to enable paid tiers/subscriptions
- Modern, Responsive UI — dark navy + terminal-green design system

2. Tech Stack

Layer	Technology
Backend	Python 3.x, Flask
Database	PostgreSQL
Frontend	HTML, CSS, JavaScript (Jinja2 templates)
Authentication	Flask-Login, bcrypt
Breach Data	HaveIBeenPwned API, BreachDirectory API
PDF Reports	ReportLab

Payments	Razorpay
Email Delivery	Brevo API (transactional) + Gmail SMTP (fallback)
Hosting (tested on)	Render

3. Requirements

- Python 3.9 or higher
- PostgreSQL 13 or higher
- pip (Python package manager)
- A HaveIBeenPwned API key (haveibeenpwned.com/API/Key)
- A Brevo account (for transactional emails) — or a Gmail account with an App Password for SMTP
- A Razorpay account (only required if enabling payments)

4. Installation Guide

Step 1 — Extract the Package

Unzip the downloaded file into your project folder.

Step 2 — Create a Virtual Environment

```
python3 -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
```

Step 3 — Install Dependencies

```
pip install -r requirements.txt
```

Step 4 — Configure Environment Variables

A file named **.env.example** is included in the root directory. Copy it and rename it to **.env**, then fill in your own values:

```
cp .env.example .env  # On Windows: copy .env.example .env
```

Your **.env** file should contain the following keys:

```
SECRET_KEY=your_secret_key_here
DATABASE_URL=postgresql://username:password@host:port/dbname
HIBP_API_KEY=your_haveibeenpwned_api_key
RAPIDKEY=your_rapidapi_key_for_breachdirectory
BREVO_API_KEY=your_brevo_api_key
MAIL_USERNAME=your_gmail_address@gmail.com
MAIL_PASSWORD=your_gmail_app_password
RAZORPAY_KEY_ID=your_razorpay_key_id
```

```
RAZORPAY_KEY_SECRET=your_razorpay_key_secret
```

Note: BREVO_API_KEY is used for transactional emails via Brevo's HTTP API. MAIL_USERNAME and MAIL_PASSWORD are used as a Gmail SMTP fallback if you prefer not to use Brevo — you only need to configure one of the two email methods. For Gmail SMTP, MAIL_PASSWORD must be a Google App Password, not your regular account password.

If you don't plan to enable payments right away, you can leave the Razorpay fields blank — the app will run without them, but the payment feature will not function until they're added.

Step 5 — Set Up the Database

This will create all required tables (users, scan_history, etc.) in your PostgreSQL database.

```
flask db upgrade
```

Step 6 — Run the App Locally

```
flask run
```

By default, the app will be available at http://127.0.0.1:5000.

5. Deployment (Render Example)

1. Push your code to a GitHub repository.
2. Create a new Web Service on Render.
3. Connect your GitHub repo.
4. Set the build command: pip install -r requirements.txt
5. Set the start command: gunicorn app:app
6. Add all environment variables from Step 4 above in the Render dashboard under Environment.
7. Create a PostgreSQL instance on Render (or use an external provider) and update DATABASE_URL accordingly.
8. Deploy — Render will build and host your app automatically.

Note on email delivery: Some free hosting tiers block outbound SMTP traffic on certain ports. This project uses Brevo's HTTP API by default to avoid this issue entirely. The Gmail SMTP option is provided as an alternative and works on most standard hosts, but if you encounter blocked SMTP ports on a free tier, switch to the Brevo API method.

6. Customization Notes

Styling: All templates use Jinja2 and are located in the /templates folder. The design system uses CSS variables for colors, defined in /static/css/style.css — update these to re-theme the app.

Breach APIs: The breach-checking logic is in /app/services/breach_check.py. You can swap or add additional breach data sources here.

Payment Tiers: Razorpay integration logic is in /app/routes/payments.py. Update pricing and plan logic here.

Monitoring Frequency: The daily re-check job is handled via a scheduled task — see /app/tasks/monitor.py to adjust frequency or logic.

Email Provider: Email sending logic is in /app/services/email_service.py. It defaults to the Brevo API and falls back to Gmail SMTP if BREVO_API_KEY is not set.

7. Known Limitations

- Account deletion (full data removal) is not yet implemented in this version — this is a planned improvement.
- Bulk scanning performance depends on the size of the CSV and API rate limits.

8. Support

This item includes documentation to help you set up and use GhostLeaks independently. For questions specific to this codebase, feel free to reach out via the contact information provided on your purchase page.

Please note that support does not cover general web hosting, server configuration, or third-party API issues (e.g. HaveIBeenPwned, Brevo, Gmail, or Razorpay account issues) — please refer to their respective documentation for those.

9. License

This item is licensed for use per the terms of your purchase agreement. Redistribution of the source code as a standalone product (with or without modification) is not permitted under the standard license.

Thank you for purchasing GhostLeaks. I hope it serves as a solid foundation for your own breach-monitoring tool or project.